5

III

Diffuse Global Illumination with Temporally Coherent Light Propagation Volumes

Anton Kaplanyan, Wolfgang Engel, and Carsten Dachsbacher

5.1 Introduction

The elusive goal of real-time global illumination in games has been pursued for more than a decade. The most often applied solution to this problem is to use precomputed data in lightmaps (e.g., Unreal Engine 3) or precomputed radiance

Figure 5.1. Example of indirect lighting with Light Propagation Volumes in the upcoming blockbuster *Crysis 2*.

transfer (e.g., *Halo 3*). Both techniques increase the complexity and decrease the efficiency of a game production pipeline and require an expensive infrastructure (e.g., setting up a cloud for precomputation and incorporating the result into a build process).

In this chapter we describe the light propagation volumes, a scalable real-time technique that does not require a preprocess and storing additional data. The basic idea is to use a lattice storing the light and the geometry in a scene. The directional distribution of light is represented using low-order spherical harmonics. The surfaces of the scene are sampled using reflective shadow maps and this information is then used to initialize the lattice for both light propagation and blocking. A data-parallel light propagation scheme allows us to quickly, and plausibly, approximate low-frequency direct and indirect lighting including fuzzy occlusion for indirect light. Our technique is capable of approximating indirect illumination on a vast majority of existing GPUs and is battle-tested in the production process of an AAA game. We also describe recent improvements to the technique such as improved temporal and spatial coherence. These improvements enabled us to achieve a time budget of 1 millisecond per frame on average on both Microsoft Xbox 360 and Sony PlayStation 3 game consoles.[1]

5.2 Overview

The light propagation volume technique consists of four stages:

- At the first stage we render all directly lit surfaces of the scene into reflective shadow maps [Dachsbacher and Stamminger 05] (see Figure 5.2).
- Next, a sparse 3D grid of radiance distributions is initialized with the surface samples generated in the first pass (see Figure 5.3).

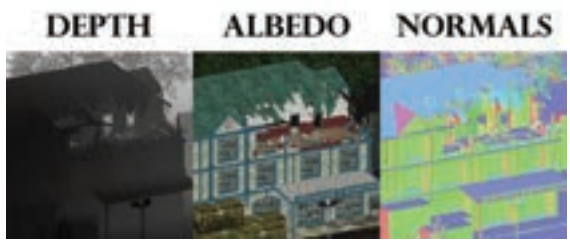

Figure 5.2. Reflective Shadow Maps store not only depth, but also information about surfaces' normals and reflected flux.

[1] *Crysis 2*, *Halo 3*, and CryENGINE 3 are trademarked. PlayStation 3, Microsoft Xbox 360, Unreal Engine 3, and Microsoft DirectX are all registered trademarks.

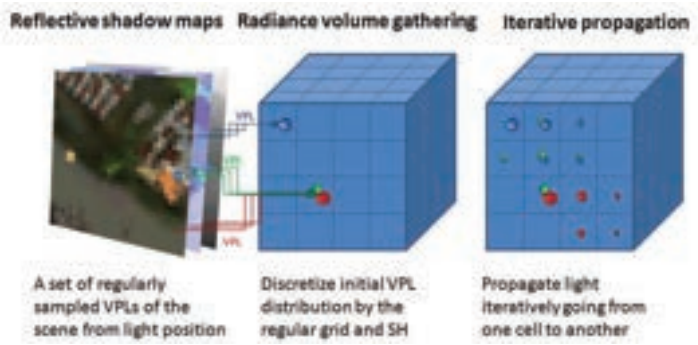

Figure 5.3. The basic steps of our method: surfaces causing one-bounce indirect illumination are sampled using RSMs, next this information is used to initialize the light propagation volumes where light is propagated, and finally this information is used to light the surfaces in the scene.

- Light is propagated in the grid using an iterative propagation scheme.
- Lastly, the scene is illuminated using the resulting grid, similar to using irradiance volumes [Tatarchuk 04].

Our light propagation volumes (LPV) technique works completely on the GPU and has very modest GPU requirements.

5.3 Algorithm Detail Description

In this section we recapitulate the core algorithm presented in [Kaplanyan and Dachsbacher 10] and emphasize several small, yet important, details. Besides the main steps of the technique we will present an important extension: cascading the reflective shadow maps. This extension allows our technique to be used in large scenes while maintaining real time performance. We also describe important general as well as platform-specific optimizations at the end of this section.

5.3.1 Reflective Shadow Maps

Reflective shadow maps (RSMs) are an extension to regular shadow maps and store not only a depth buffer, but also a normal buffer and flux buffer (Figure 5.2). It is a very fast method for sampling directly lit surfaces of a scene on the GPU, and all pixels of such an RSM can be seen as virtual light sources that generate the indirect illumination in a scene. This representation allows generating and storing samples of a scene's lit surfaces in a very efficient manner.

The size of an RSM should be chosen such that one surface sample (*surfel*) represents an area that is much smaller than one cell of the LPV to provide

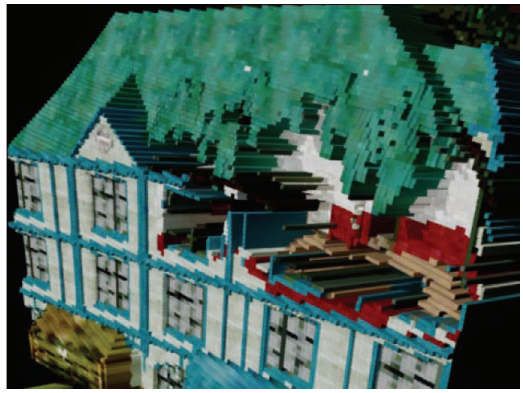

Figure 5.4. A reflective shadow map captures the directly lit surfaces of the scene with a regular sampling from the light's origin.

sufficient detail in the lighting computation. We recommend an RSM size that is four times larger than the number of elements along the diagonal of an LPV (e.g., for a LPV of $32 \times 32 \times 32$, an RSM of at least 128×128 size is recommended).

As in the original approach, we assume indirect lighting from diffuse surfaces and store reflected flux in the RSM, which accounts for surface albedo as well as incident lighting (i.e., effects such as a colored projected light can be used as well).

Note that many techniques developed for shadow maps can be applied to RSMs. For example, an RSM can be cascaded like cascaded shadow maps to capture global illumination from a light source such as the sun, or it can be stored in cube maps similar to which cube shadow maps for point light sources. Figure 5.4 shows the surfels created from an RSM that is created for a directional light source.

Temporally stable rasterization. Temporal flickering can occur when an RSM frustum moves, and obviously becomes visible when the resolution of the RSM is rather low and thus scene surfaces are sampled at a coarser level. A simple solution to this problem is to move the frustum of the RSM with world-space positions snapped to the size of one texel in world space. This leads to consistently sampled points during rasterization and largely removes the sampling problems [Dimitrov 07]. If this is not possible, e.g. for perspective projections, then higher resolution RSMs are required for consistently sampling surfaces. Kaplanyan et al. [Kaplanyan 09] proposed downsampling RSMs to reduce the number of surfels for the injection stage. However, additional experiments showed that the downsampling if sometimes even slower than injecting the same number of surfaces directly into the LPV.

5.4 Injection Stage

The injection stage transforms the reflected flux of the surfels obtained from the RSMs into an initial light distribution represented using spherical harmonics (SH) stored into the LPV as spatial discretization. As we assume diffuse surfaces, the reflected flux of each surfel (neglecting if spatial extend) can be represented using a cosine lobe and thus using a low-order SH approximation. Note that we store the energy as directional intensity distribution in the LPV as described in [Kaplanyan and Dachsbacher 10].

The LPV is stored as a texture on the GPU and thus the SH approximations of the surfels can be easily accumulated into the LPV using additive blending. At this point the spatial discretization comes into play as the surfels' positions are snapped to the centers of LPV cells (see Figure 5.5). At the end of the injection stage, each cell of the LPV represents the initial radiant intensity approximated using spherical harmonics.

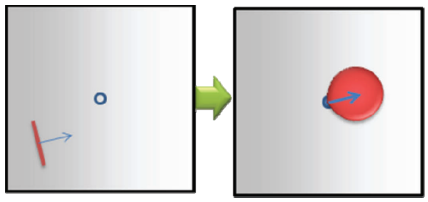

Figure 5.5. The reflected intensity distribution of a surfel obtained from an RSM is approximated using SH and snapped to the center of the closest LPV cell.

Projecting into spherical harmonics. To represent the intensity distribution of a surfel we use the first two bands of the spherical harmonics basis (i.e., four coefficients). This proved to be sufficient to represent the mostly low-frequency indirect lighting in diffuse scenes (Figure 5.6). As previously mentioned, the outgoing intensity distribution of each surfel is a (clamped) cosine lobe centered around its surface normal that is projected into the SH basis. Given the normal vector of the surfel, $\mathbf{n}$, we can obtain the SH coefficients $\mathbf{c} = (c_0, c_1, c_2, c_3)$ as [Sloan 08]:

$$
\begin{aligned}
c_0 &= \frac{\sqrt{\pi}}{2}, \\
c_1 &= -\sqrt{\frac{\pi}{3}}y, \\
c_2 &= \sqrt{\frac{\pi}{3}}z, \\
c_3 &= -\sqrt{\frac{\pi}{3}}x.
\end{aligned}
$$

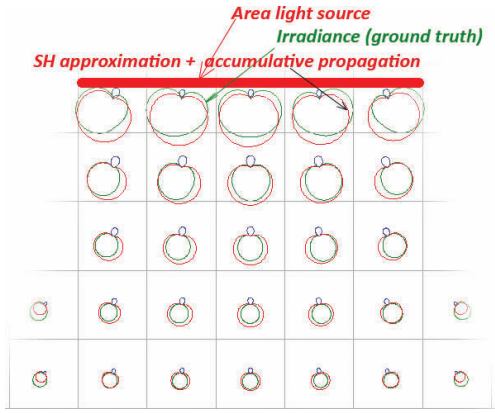

Figure 5.6. Illustration of the approximation error with 4 SH coefficients and a coarse lattice. Note that both the analytical result (green) and propagated result (red) are represented as a final convolved irradiance.

These coefficients are scaled according to the corresponding world-space size and reflected flux of the surfel, yielding four coefficients per color channel. In order to simplify the following description, we show only one set of SH coefficients.

Offsetting surfels. As the LPV is a coarse grid, we have to take care when injecting surfels, as their exact position inside a cell is no longer available after injection. If a surfel's normal points away from the cell's center, its contribution should not be added to this cell, but rather to the next cell, in order to avoid self-illumination (see Figure 5.7). For this, we virtually move each VPL by half the cell-size in the direction of its normal before determining the cell. Note that this shifting of the surfels still does not guarantee completely avoiding self-illumination and light bleeding, but largely removes artifacts from the injection stage.

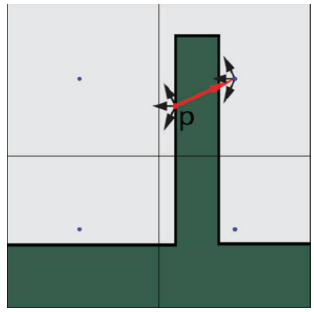

Figure 5.7. Example of a VPL injection causing self-illumination of the thin geometry.

5.4.1 Propagation

The propagation stage consists of several iterations, where every iteration represents one step of light propagation. The propagation stage has some similarity to the *SH discrete ordinate method* [Chandrasekhar 50, Evans 98]. These techniques are typically employed for light-propagation simulation in scattering media. Essentially we use the same process; however, we use a different cell-to-cell propagation scheme. The application of this method to light propagation through vacuum instead of scattering media suffers from the fact that propagation directions are blurred out. Fortunately, our results show that this is an acceptable artifact in many application scenarios.

Intensity propagation. The propagation consists of the following steps:

- The input for the first iteration step is the initial LPV obtained from the injection stage. Each cell stores the intensity as an SH-vector and we propagate the energy to the six neighbors along the axial directions.
- All subsequent propagation steps take the LPV from the previous iteration as input and propagate as in the first iteration.

The main difference from SHDOM methods is the propagation scheme. Instead of transferring energy from a source cell to its 26 neighbor cells in a regular grid, we propagate to its 6 neighbors only, thus reducing the memory footprint. To preserve as much directional information as possible, we compute the transfer to the faces of these neighbor cells and reproject the intensities to the cells' center (see Figure 5.8). This mimics, but is of course not identical to, the use of 30 unique propagation directions. Please see [Kaplanyan and Dachsbacher 10] for the details. There are two ways to implement this propagation process: scattering and gathering light. The gathering scheme is more efficient in this case due to its cache-friendliness.

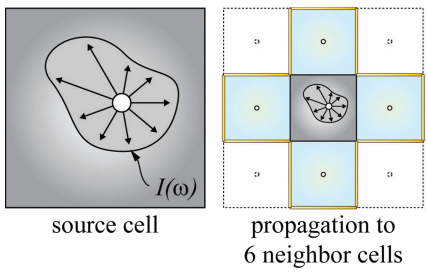

Figure 5.8. Propagation from one source cell (center) to its neighbor cells. Note that we compute the light propagation according to the intensity distribution $I(\omega)$ to all yellow-tagged faces of the destination cells (blueish).

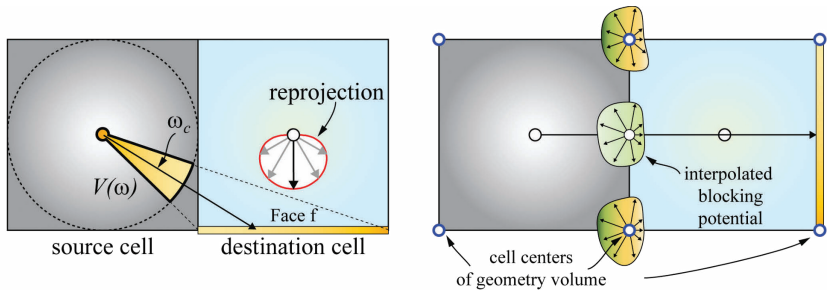

Figure 5.9. Left: Light propagation from the source cell (gray) to the bottom face of the destination cell (blueish). Right: during propagation we can account for occlusion by looking up the blocking potential from the geometry volume.

When propagating the light from a source cell to one face of the destination cell, we compute the incoming flux onto the face using the solid angle $\Delta\omega$ of the face and the central direction ω_c of the propagation cone (see Figure 5.9). The flux reaching the face is then computed as

$$\Phi_f = \frac{\Delta\omega}{4\pi} I(\omega_c),$$

where $I(\omega_c)$ is the intensity of the source cell towards the center of face obtained by evaluating the SH approximation. Here we assume that the intensity does not vary over the solid angle of the face.

Reprojection. The flux incident on a face is then reprojected back into the intensity distribution of the destination cell. The reprojection is accomplished by creating a new virtual surfel at the destination cell's center, pointing toward the face and emitting exactly as much flux as the face received from the propagation (Φ_f):

$$\Phi_l = \int_\Omega \Phi_f \langle n_l, \omega \rangle d\omega = \frac{\Phi_f}{\pi}.$$

Similar to the light injection stage, the corresponding clamped cosine lobe is scaled by Φ_l and accumulated into SH coefficients of the destination cell. In other words, we compute the incoming flux for each face of the destination cell and transform it back into an intensity distribution.

Blocking for indirect shadows. Indirect shadows, i.e., the blocking of indirect light due to scene geometry, can also be incorporated into the LPVs. In order to add indirect shadows, we construct a volumetric representation of the scene's surfaces (see Section 5.4.2). This so-called geometry volume (GV) is a grid of the same resolution as the LPV and stores the blocking potential (also represented as SH)

for every grid cell. The GV is displaced by half the grid size with respect to the LPV. That is, the cell centers of the GV reside on the corners of the LPV cells. Whenever we propagate from a source to a destination cell, we obtain the bilinearly interpolated SH coefficients—at the center of the face through which we propagate—from the GV, and evaluate the blocking potential for the propagation direction to attenuate the intensity. Note that this occlusion should not be considered for the very first propagation step after injection, in order to prevent immediate self-shadowing.

Iterations. The sum of all intermediate results is the final light distribution in the scene. Thus we accumulate the results of every propagation in the LPV into a separate 3D grid. The number of required iterations depends on the resolution of the volume. We recommend using two times the longest dimension of the grid (when not using a cascaded approach). For example, if the volume has dimensions of $32 \times 32 \times 8$, then the light can travel the whole volume in 64 iterations in the worst case (which is a diagonal of the volume). However, when using a cascaded approach it is typically sufficient to use a significantly lower number of iterations.

Illustrative example of light propagation process in the Cornell Box–like scene. The light propagation is shown in Figure 5.10. The top-left image shows the coarse LPV initialized from the RSM in the injection stage. The noticeable band of reflected blue and red colors has a width of one cell of LPV. Note that after four iterations the indirect light is propagated and touches the small white cube. After eight iterations the indirect light has reached to the opposite wall.

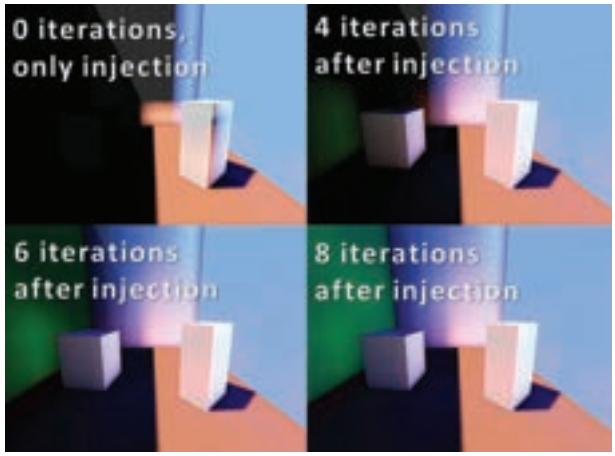

Figure 5.10. Example of the light propagation process in a simple scene.

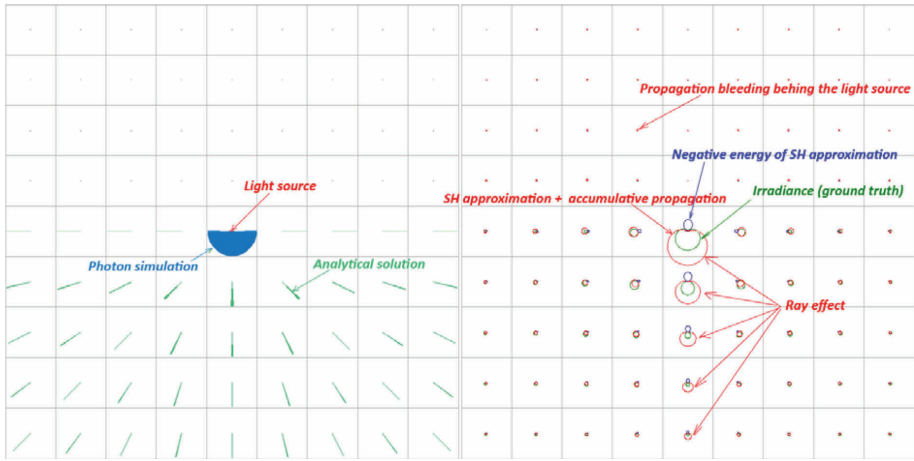

Figure 5.11. Light from a small area light source (smaller than the cell size). The analytical result shows the ground truth solution (left). The light propagation illustrates the ray effect and an unwanted propagation behind the light source (right).

Limitations. The iterative propagation has three main limitations. First, the coarse resolution of the grid does not capture fine details and might even cause light bleeding. Second, due to the SH representation of the intensity distribution and the propagation process itself, the light strongly diffuses and a strictly directed light propagation is not possible. Consequently, there is also no reasonable chance of handling glossy surfaces during propagation. Lastly, the propagation together with the reprojection introduces spatial and directional discretization; this is called the ray effect and is common to all lattice-based methods (see Figure 5.11). Note that some of these artifacts can be suppressed to an acceptable level using cascaded grids to provide finer grids closer to the camera and smart filtering when looking up the LPV.

5.4.2 Indirect Occlusion (Optional Step for Indirect Shadows)

The geometry volume storing the blocking potential of the scene geometry is only required when computing indirect shadows. To this end, we again sample the surfaces using RSMs (and deferred shading buffers, see below) and each texel represents a small part of a surface. Using this sampled scene information, we model the occlusion in the spirit of [Sillion 95], using the accumulated blocking potential of surfels in a grid cell as a probability for blocking light from a certain direction. In this way we can render soft shadows, but shadows of objects smaller than the GV's cell size can of course not be resolved.

Constructing the geometry volume. The blocking potential of a surfel is approximated using the first two bands of the SH basis again. The amount of blocking by one surfel depends on its size and the cosine of the angle between its normal and the light direction in question. The blocking probability of a single surfel with area A_s (computed based on the RSM's texel size in world space), and normal n_s in a cell of grid size s is

$$B(\omega) = \frac{A_s \langle n_s | \omega \rangle}{s^2}.$$

Note that we assume that scene objects are closed surfaces. This is important and allows us to use a clamped cosine lobe for the blocking potential again, as low-order SH projections of absolute cosine lobes tend to degrade to near isotropic functions. Similar to the VPL injection, we accumulate the SH projections of the blocking potential into the GV.

Reusing G-buffers from RSMs and cameras. Aiming at fully dynamic scenes without precomputation requires the creation of the GV—and thus the surface sampling—on the fly. First of all, we can reuse the sampling of the scene's surfaces that is stored in the depth and normal buffers of the camera view (when using a deferred renderer), and in the RSMs that have been created for the light sources. RSMs are typically created for numerous light sources and thus already represent a dense sampling of large portions of the scene. It is, at any time, possible to gather more information about the scene geometry by using depth-peeling for the RSMs or the camera view.

The injection (i.e., determining the blocking potential of a GV's cell) has to be done using separate GVs for every RSM or G-buffer in order to make sure that the same occluder (surfel) is not injected multiple times from different inputs. Afterwards, we combine all intermediate GVs into a single final GV. We experimented using the maximum operator for SH coefficients. Although this is not correct, it yields plausible result when using 2 SH-bands only and clamping the evaluation to zero, and thus yields more complete information about light-blocking surfaces.

5.4.3 Final Scene Illumination

The accumulated results of the propagation steps represent the light distribution in the scene. In the simplest form we query the intensity by a trilinearly interpolated lookup of the SH coefficients and compute the reflected radiance of a surface. We then evaluate the SH-intensity function for the negative normal of the surface, similar to irradiance volumes [Greger et al. 97], and next convert it into incident radiance. For this we assume that the cell's center (where the intensity is assumed to reside) is half the grid size s away from the surface.

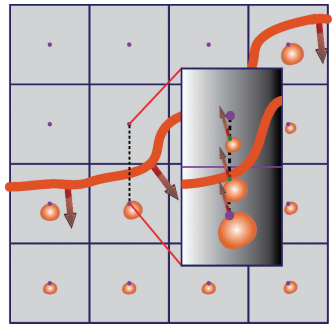

Figure 5.12. The interpolation of the intensity stored in the grid induced by the orange surface between two cells. Note the wrong result of linear interpolation behind the emitting object itself. Also note that the gradient is opposing in this case.

LPVs and light prepass rendering. In light prepass architecture [Engel 09] of the CryENGINE 3, the LPV lighting is directly rendered into the diffuse-light-accumulation buffer on top of multiple ambient passes. This allows us to use optimizations, such as stencil prepass and depth-bound tests, for this pass as well. Moreover, it is also possible to compose complex layered lighting.

Improving filtering by using directional derivatives. The trilinear interpolation of SH coefficients for looking up the LPV can cause serious artifacts such as self-illumination and light bleeding. We found that a damping factor based on the directional derivative of the intensity distribution greatly reduces these artifacts. For a surface location x with normal n, we determine the trilinearly interpolated SH coefficients c and the directional derivative in the normal direction ∇c (computed via final differencing) (see Figure 5.12):

$$\nabla c(x) = \frac{c(x) - c(x+n)}{||n||} = c(x) - c(x+n).$$

Whenever the derivative is large, and c and ∇c are deviating, we damp c before computing the lighting.

This additional filtering during the final rendering phase yields sharper edges between lit and shadowed areas.

5.4.4 A Multi-Resolution Approach using Cascaded Light Propagation Volumes

So far we have considered a single LPV only; however, a regular 3D grid does not provide enough resolution everywhere in scenes with large extent, or otherwise requires prohibitively high memory storage and propagation cost. In this section

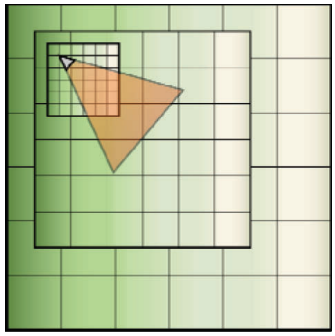

Figure 5.13. Cascaded approach. Cascades are nested and attached to the camera; note that cascades are slightly shifted toward the view direction.

we propose a multi-resolution approach to this problem keeping both memory consumption and computation cost low, even for large scenes (see Figure 5.13).

Cascaded LPVs. In spirit of the cascaded shadow map (CSM) approach [Engel 05, Dimitrov 07], we use multiple nested grids, or cascades, for light propagation that move with the camera. For every cascade, we not only store an LPV, but we also create a separate RSM for every light source, where the RSM resolution is chosen proportional to the grid cell sizes as described above. However, unlike CSMs, the indirect lighting can also be propagated from behind to surfaces in front of the camera (see Figure 5.14). In practice we use a 20/80 ratio for cascade shifting, i.e., the cascades are not centered around the camera but shifted in view directions such that 20% of their extent is located behind the camera. Usually it is sufficient to use three nested grids with a respective double size.

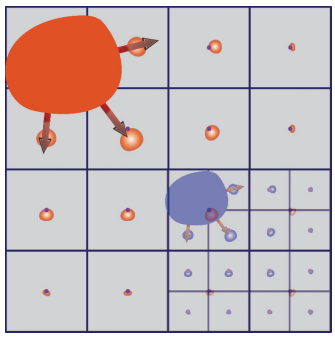

Figure 5.14. Indirect light propagated from objects of different sizes in the respective resolution.

The idea of LPVs naturally transfers to the multi-resolution approach, and every cascade requires RSM generation, injection, propagation, and LPV lookup almost as before. However, before the RSM generation starts, we determine for every object in the scene to which cascade it belongs, and thus in which RSM it has to be considered. This ensures that we do not account for the indirect light contribution of an object multiple times.

Light propagation with cascades. So far we detailed light propagation within a single grid, but handling multiple cascades simultaneously imposes new questions: How do we propagate across different cascades? How do we combine the contributions of different cascades for the final rendering? In this section we propose two options depending on whether indirect shadows are a required feature or not.

If no indirect shadows are required, we can handle the cascades independent from each other, and the multi-resolution approach is straightforward to implement. With indirect shadows, we have to correctly deal with light propagating across the edges of cascades and, of course, blocking.

Cascaded LPVs without indirect shadows. Assuming that light propagates without blocking, we can completely decouple the cascades and compute a LPV solution with the following steps:

- Every RSM for a each cascade should contain unique objects causing indirect light. Objects are normally rendered into the RSM for the cascade for which they have been selected; in RSMs for other cascades they are rendered with black albedo in order to prevent indirect light contribution, but correctly rendering direct shadows.
- The propagation is done for each cascade independently.
- The scene is illuminated by the accumulated contribution of all cascades.

In this case, we determine the respective cascade for every object by estimating its contribution to the indirect illumination, which in turn heavily depends on its surface area and the distance to the camera. To this end, we account for large and distant objects in the coarser cascades while injecting smaller, close objects into the finer grids. Note that this means that distant, small objects might not be considered during the computation (if they lie outside the finer cascades). However, the indirect illumination stemming from such objects typically has a significant contribution only within a certain (small) proximity of the object.

Cascades with indirect shadows. When accounting for light blocking we cannot decouple the propagation process of the cascades. In this case every object is injected into the finest grid at its respective location. This also means that those parts of coarser grids that are overlapped by finer grids are empty and not used during the propagation. Although we apply the propagation steps to each

cascade separately, we have to make sure that light leaving one grid can further propagate in the next grid. This is simple when the grid cell sizes of two cascades differ by a power-of-two factor. Note that intensity has to be redistributed when propagating light from a single cell of a coarse grid to multiple cells of a fine grid, and accumulated when propagating light from multiple fine grid cells to a coarse grid cell (as intensity already accounts for the surface area).

Stable lighting through snapping. The LPV cascades are oriented along the world space axes and do not rotate, only translate, with the camera. It is important to snap the cascade positions in world-space to multiples of the cell sizes. This ensures that the distribution of surfels in the injection stage is stable and independent of the translation of the LPVs.

5.5 Optimizations

5.5.1 General Optimizations

The scene illumination stage with a final LPV is usually a very expensive pass (see the timing table in Section 5.6.1). It is important to note that the hardware capability to render into a volume texture tremendously improves performance at this stage. This simplifies the shader workload as the emulation of a trilinear filtering in the pixel shader is not necessary. Instead, the hardware unit is utilized and cache coherency is highly improved due to the optimized memory layout of a swizzled 3D texture.

Unfortunately, not every platform supports rendering into a 3D texture. However, it is possible to inject the VPLs directly into the volume texture on consoles. To do so, the 3D texture should be treated as a horizontally unwrapped 2D render target; note that this is not possible with the Microsoft DirectX 9.0 API.

Using an 8 bit/channel texture format for the LPV has proven to be sufficient for diffuse indirect lighting stored in two bands of SH. Note that this detail is very important because it speeds up the final rendering pass significantly due to decreased bandwidth and texture-cache misses.

5.5.2 Temporal Coherence

As the diffuse indirect lighting is inherently smooth, the observer is usually quite unaware of temporally changing smooth gradients. Exploiting this fact, we recommend caching each resulting LPV and reusing the results across several frames. The temporal reprojection can easily be done in a straightforward way with accumulation of the old results and a smooth substitution by the newly propagated results.

In practice, we perform the full pass of LPV computation (RSM generation, injection, and propagation) each fifth frame. With the target frame rate of 30 frames/second that means that the refresh rate for the indirect lighting is six

times/second. This proved to be sufficient in the vast majority of cases. The workload can be distributed either stage-by-stage across several frames or can be executed all at once (RSM rendering, injection, and propagation) during one frame. In CryENGINE 3 we do not spread separate stages; however, we update different cascades in different frames. In addition, we use different update frequencies for different cascades of cascaded shadow maps. The LPV update workload is interleaved with updates of shadow map cascades, which ensures that we have a balanced rendering and consistent performance.

5.5.3 Xbox 360-Specific Optimizations

When doing the injection directly into a 3D texture on the Xbox 360, we recommend redundantly unwrapping this texture vertically as well (so a $32 \times 32 \times 32$ 3D texture becomes a 32×1024 2D texture after being aliased as a 2D render target, but it should additionally be duplicated vertically). This trick significantly reduces the h/w bank conflicts caused by multiple surfels being written to the same pixel.

Note that the redundant copies of the texture should be accumulated together afterwards and added into the final 3D texture. Also, the resolve GPU command from the EDRAM into a 3D texture has some API issues; a work-around solution for these issues is proposed in the Appendix of [Kaplanyan 09].

5.5.4 PlayStation 3-Specific Optimizations

The 3D texture is essentially implemented as an array of contiguous 2D textures. This means that the layout of unwrapped 2D render targets should be vertical on PlayStation 3 in order to match the memory layout of the 3D texture. Then the injection goes directly into a 3D texture and can be done by using the simple memory aliasing of this texture as a 2D render target. The RSX pipeline should obviously be flushed after the injection rendering in order to make sure that all the surfels are injected before the propagation is started.

Another suggestion for speeding up the final illumination pass is to alias the target buffer as a $2\times$ MSAA and do the shading in a half-resolution horizontally. This is possible only on PlayStation 3 because the layout of samples in the sampling scheme matches the layout of the render target with the same pitch. This trick can help to reduce the pixel work without noticeable quality degradation, as the diffuse indirect lighting typically exhibits low frequencies only.

5.6 Results

In this section we provide several example screenshots showing diffuse global illumination in Crysis 2 (see Figure 5.15). In Crysis 2 we use one to three cascades depending on the graphics settings, platform, and level specifics.

Figure 5.15. In *Crysis 2* we use from one to three cascades depending on graphics settings, platform, and level specifics.

5.6.1 Tools and Tweaking Parameters for Artists

Because artists want to have more control over the indirect lighting, it was decided to expose multiple controlling parameters for the technique:

- *Global intensity of indirect lighting.* This parameter helps to tweak the overall composition and can also be tweaked during the game play from the game logics.
- *Per-object indirect color.* This parameter affects only the color and the intensity of indirectly bounced lighting from a specific object. It is mostly used to amplify or attenuate the contribution of some particular objects into indirect lighting.
- *Propagation attenuation.* This parameter is used to tweak the attenuation of the indirect color.
- *Per-object "indirect receiver" parameter for particles.* This is mostly used to tag some particle effects as those illuminated by the indirect lighting (disabled by default).

These parameters are frequently used by artists to tweak some particular places and moments in the game. They proved to be particularly important for cut scenes and some in-game movies.

5.6.2 Timings

Detailed timings for a single cascade for the Crytek Sponza[2] scene are provided in Table 5.1. Note that the timings for multiple cascades can be estimated by multiplying the timings for a single cascade by the number of cascades when using the cascaded approach, as the work is spread across several RSMs.

For all screenshots in this chapter we used the same settings: the size of the LPV grid is $32 \times 32 \times 32$, the propagation uses 12 iterations and 1 1 cascade, and the rendering was at 1280×720 resolution (no MSAA). The cost of the final illumination pass obviously depends on the target buffer resolution. The RSM size is 256^2 for the timings with the NVIDIA GTX285 and 128^2 for both consoles. Of course the cost of the RSM rendering also depends on the scene complexity.

Stage	**NVIDIA GTX 285**	**Xbox**	**PlayStation**
RSM Rendering	0.16 (256^2)	0.5 (128^2)	0.8 (128^2)
VPL Injection	0.05	0.2	0.4
Propagation (12 iterations)	0.8	0.8	0.7
Final illumination (720p)	2.4 (RGBA16F format)	2.0	1.5
Total (per frame)	**1.5**	**1.1**	**1.2**

Table 5.1. Detailed timings for a single cascade for the Crytek Sponza scene. All measurements are in milliseconds for the individual stages.

5.7 Conclusion

In this chapter we described a highly parallel production-ready and battle-tested (diffuse) global illumination method for real-time applications without any pre-computation. To our knowledge, it is the first technique that employs a light propagation for diffuse global illumination. It features a very consistent and scalable performance, which is crucial for real-time applications such as games. We also described how to achieve indirect illumination in large-scale scenes using a multi-resolution light propagation.

We demonstrated our method in various scenes in combination with comprehensive real-time rendering techniques. In the future we would like to reduce the limitations of our method by investigating other grid structures and other adaptive schemes, where the compute shaders of DirectX 11 will probably of great help.

[2]The Crytek Sponza scene is the original Sponza Atrium scene improved for global illumination experiments. This scene is granted to the rendering community and can be downloaded from this link: http://crytek.com/cryengine/cryengine3/downloads.

5.8 Acknowledgments

Thanks to Sarah Tariq and Miguel Sainz for implementing this technique as a sample of the NVIDIA SDK[3]. And of course the whole Crytek R&D team as well as all others who helped and discussed the real-time graphics with us!

Bibliography

[Chandrasekhar 50] S. Chandrasekhar. *Radiative Transfer*. New York: Dover, 1950.

[Dachsbacher and Stamminger 05] C. Dachsbacher and M. Stamminger. "Reflective Shadow Maps." In *Proc. of the Symposium on Interactive 3D Graphics and Games*, pp. 203–213. Washington, DC: IEEE Computer Society, 2005.

[Dimitrov 07] R. Dimitrov. "Cascaded Shadow Maps." Technical report, NVIDIA Corporation, 2007.

[Engel 05] W. Engel. "Cascaded Shadow Maps." In *Shader X*[5], pp. 129–205. Hingham, MA: Charles River Media, 2005.

[Engel 09] W. Engel. "Light Prepass." In *Advances in Real-Time Rendering in 3D Graphics and Games Course—SIGGRAPH 2009*. New York: ACM Press, 2009.

[Evans 98] K. F. Evans. "The Spherical Harmonic Discrete Ordinate Method for Three-Dimensional Atmospheric Radiative Transfer." In *Journal of Atmospheric Sciences*, pp. 429–446, 1998.

[Greger et al. 97] G. Greger, P. Shirley, P. Hubbard, and D. Greenberg. "The Irradiance Volume." *IEEE Computer Graphics & Applications* 18 (1997), 32–43.

[Kaplanyan and Dachsbacher 10] Anton Kaplanyan and Carsten Dachsbacher. "Cascaded Light Propagation Volumes for Real-Time Indirect Illumination." In *Proceedings of the 2010 ACM SIGGRAPH Symposium on Interactive 3D Graphics and Games*, pp. 99–107, 2010.

[Kaplanyan 09] A. Kaplanyan. "Light Propagation Volumes in CryEngine 3." In *Advances in Real-Time Rendering in 3D Graphics and Games Course—SIGGRAPH 2009*. New York: ACM Press, 2009.

[Sillion 95] F. Sillion. "A Unified Hierarchical Algorithm for Global Illumination with Scattering Volumes and Object Clusters." *IEEE Trans. on Visualization and Computer Graphics* 1:3 (1995), 240–254.

[Sloan 08] P.-P. Sloan. "Stupid Spherical Harmonics (SH) Tricks." In *GDC'08*, 2008.

[Tatarchuk 04] N. Tatarchuk. "Irradiance Volumes for Games." Technical report, ATI Research, Inc., 2004.

[3]Publicly available for download at: http://developer.nvidia.com/object/sdk_home